زمانی میان آغاز یک قله و شروع قله‌ی بعدی، انتقال از یک مجموعه‌ی کاری به مجموعه‌ی کاری دیگر را نشان می‌دهد.

۱۰-۶-۳ فرکانس خطای صفحه

مدل مجموعه‌ی کاری موفقیت‌آمیز است و دانش مجموعه‌ی کاری هم می‌تواند برای پیش‌بند صفحه (بخش ۱۰-۹-۱) مفید باشد، اما به نظر می‌رسد که برای کنترل کوبیدگی، روش پخته‌ای نیست. یک راهبرد که از **فرکانس خطای صفحه**[1] **(PFF)** استفاده می‌کند رویکرد مستقیمی در پیش می‌گیرد.

مسأله‌ی به‌خصوص آن است که چگونه از کوبیدگی پیشگیری کنیم. کوبیدگی نرخ خطای صفحه‌ی بالایی دارد. بنابراین، می‌خواهیم نرخ خطای صفحه را کنترل کنیم. موقعی که خیلی بالاست، می‌دانیم که پردازه به قاب‌های بیشتری نیاز دارد. برعکس، اگر نرخ خطای صفحه زیادی پایین باشد، در اینجا ممکن است پردازه قاب‌های بسیار زیادی در اختیار داشته باشد. می‌توانیم روی نرخ خطای مطلوب، یک کران بالا و پایین بگذاریم (شکل ۱۰-۲۳). اگر نرخ خطای صفحه از حد بالا تجاوز نماید، قاب دیگری به پردازه تخصیص می‌دهیم. اگر نرخ خطای صفحه به زیر حد پایین‌تر برسد، یک قاب از پردازه بر می‌داریم. بنابراین، برای پیشگیری از کوبیدگی می‌توانیم نرخ خطای صفحه را به طور مستقیم سنجیده و کنترل کنیم.

همان‌گونه که در راهبرد مجموعه‌ی کاری داشتیم، ممکن است به ناچار پردازه‌ای را به خارج از حافظه مبادله کنیم. اگر نرخ خطای صفحه افزایش یابد و هیچ قاب آزادی نباشد، بایستی پردازه‌ای را انتخاب کرده و به ذخیره‌ی پشتیبان مبادله کنیم. در ادامه، قاب‌های آزاد شده در میان پردازه‌هایی که نرخ خطای صفحه‌ی بالایی دارند، توزیع می‌شوند.

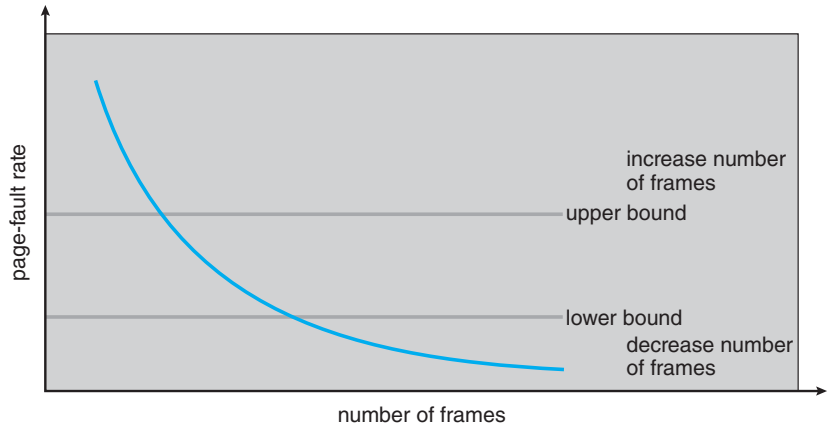
۱۰-۶-۴ مشق فعلی

به بیان عملی، کوبیدگی و مبادله‌ی حاصل به‌واقع یک پیامد ناخوشایند گرانی بر کارایی دارند. بهترین مشق[2] فعلی در پیاده‌سازی یک سیستم کامپیوتری این است که هر جا مقدور باشد، برای اجتناب از کوبیدگی و مبادله، حافظه‌ی فیزیکی کافی لحاظ شود. از گوشی‌های هوشمند گرفته تا کارگزارهای بزرگ، تدارک حافظه‌ی کافی برای حفظ تمامی مجموعه‌ی کاری به صورت همروند در حافظه، مگر در شرایط استثناء، بهترین تجربه را برای کاربر فراهم می‌کند.

۱۰-۷ فشرده‌سازی حافظه

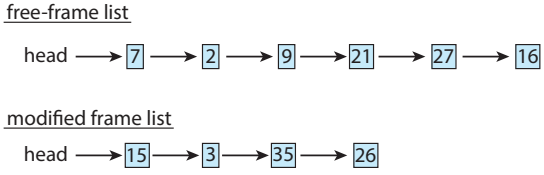
یک جایگزین برای صفحه‌بندی، **فشرده‌سازی حافظه** است. در اینجا، به‌جای این که قاب‌های تغییریافته به فضای مبادله برده شوند، چندین قاب را به‌شکل یک قاب منفرد فشرده می‌کنیم، این کار سیستم را قادر می‌سازد تا استفاده از حافظه را کاهش بدهد بدون این که به مبادله‌ی صفحات اقدام کند.

1. Page-fault frequency
2. Practice



شکل ۱۰-۲۳ فرکانس خطای صفحه.

در شکل ۱۰-۲۴، لیست قاب آزاد شامل شش قاب است. فرض کنید که این تعداد قاب آزاد، زیر آستانه‌ی معین است که جایگزین صفحه را راه می‌اندازد. الگوریتم جایگزینی (برفرض، یک الگوریتم تقریب LRU) چهار قاب—۱۵، ۳، ۳۵، و ۲۶—را برای گذاشتن در لیست قاب آزاد انتخاب می‌کند. ابتدا این قاب‌ها را در یک لیست قاب تغییریافته قرار می‌دهد. لیست قاب تغییریافته نوعاً در ادامه در فضای مبادله نوشته خواهند شد که قاب‌ها را در دسترس لیست قاب آزاد قرار می‌دهد. یک راهبرد جایگزین، فشرده‌سازی تعدادی از قاب‌ها—برفرض، سه تا—و ذخیره‌سازی نسخه‌های فشرده‌ی آن‌ها در یک قاب منفرد می‌باشد.

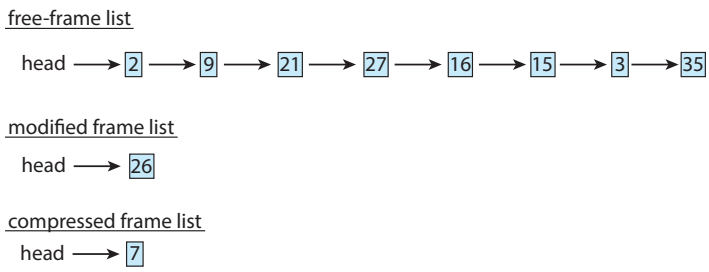


شکل ۱۰-۲۴ لیست قاب آزاد قبل از فشرده‌سازی.

در شکل ۱۰-۲۵، قاب ۷ از لیست قاب آزاد برداشته می‌شود. قاب‌های ۱۵، ۳، و ۳۵ فشرده شده و در قاب ۷ ذخیره می‌شوند که در ادامه در لیست قاب‌های فشرده ذخیره می‌گردد. قاب‌های ۱۵، ۳، و ۳۵ هم‌اکنون به لیست قاب آزاد منتقل می‌شوند. اگر بعداً یکی از سه قاب فشرده مراجعه داشته باشند، یک خطای صفحه اتفاق می‌افتد، و قاب فشرده از حالت فشرده در می‌آید و سه قاب ۱۵، ۳، و ۳۵ در حافظه بازیابی می‌شوند.

همان‌گونه که متذکر شدیم، سیستم‌های موبایل عموماً از مبادله‌ی استاندارد و یا مبادله‌ی صفحات پشتیبانی نمی‌کنند. بنابراین، فشرده‌سازی حافظه یک بخش جامع راهبرد مدیریت حافظه برای سیستم‌عامل‌های موبایل، اعم از اندروید و iOS می‌باشد. به‌علاوه، هر دوی ویندوز ۱۰ و macOS از فشرده‌سازی حافظه پشتیبانی می‌کنند. در ویندوز، مایکروسافت معماری **سکوی جهانی**

ویندوز[1] **(UWP)** را توسعه داده است که یک سکوی app رایجی برای دستگاه‌هایی که ویندوز ۱۰ را اجرا می‌کنند، اعم از دستگاه‌های موبایل فراهم می‌کند. appهای UWP که بر روی دستگاه‌های موبایل اجرا می‌شوند نامزد فشرده‌سازی حافظه هستند. macOS نخست از نسخه‌ی 10.9 سیستم‌عامل از فشرده‌سازی حافظه پشتیبانی کرد، موقعی که حافظه‌ی آزاد کم می‌شود صفحات LRU را در ابتدا فشرده می‌کند و سپس اگر مسئله حل نشود از صفحه‌بندی استفاده می‌کند. آزمون‌های کارایی نشان می‌دهد که فشرده‌سازی حافظه حتی سریع‌تر از صفحه‌بندی حافظه‌ی ثانویه‌ی SSD در لپ‌تاپ و سیستم‌های macOS رومیزی است.



شکل ۱۰-۲۵ لیست قاب آزاد بعد از فشرده‌سازی.

اگرچه فشرده‌سازی حافظه مستلزم تخصیص قاب‌های آزاد برای حفظ قاب‌های فشرده است، بسته به کاهش‌های حاصل از الگوریتم فشرده‌سازی، یک صرفه‌جویی قابل توجهی می‌تواند محقق شود. (در مثال بالا، سه قاب به یک‌سوم اندازه‌ی اصلی آن‌ها کاهش می‌یابند.) مثل هر شکل فشرده‌سازی داده‌ها، یک رقابتی میان سرعت الگوریتم فشرده‌سازی و میزان کاهش بدست آمده وجود دارد (تحت عنوان **ضریب فشرده‌سازی**). در حالت کلی، ضرایب فشرده‌سازی بالاتر (کاهش‌های بیشتر) می‌تواند با الگوریتم‌های کندتر با عطش محاسبات بیشتر بدست آید. اکثر الگوریتم‌هایی که امروزه استفاده می‌شوند این دو عامل را موازنه می‌سازند، دست‌یابی به ضرایب فشرده‌سازی بالا با استفاده از الگوریتم‌های پرسرعت. به‌علاوه، الگوریتم‌های فشرده‌سازی با بهره‌مندی از هسته‌های پردازشی متعدد و انجام موازی فشرده‌سازی بهبود پیدا کرده‌اند. برای مثال، الگوریتم‌های فشرده‌سازی Xpress مایکروسافت و WKdm اَپل سریع محسوب می‌شوند و فشرده‌سازی صفحات را تا ۳۰ تا ۵۰ درصد اندازه‌ی اصلی آن‌ها گزارش داده‌اند.

۱۰-۸ تخصیص حافظه‌ی هسته

زمانی که پردازه‌ای با اجرا در مد کاربر حافظه‌ی بیشتری درخواست می‌کند، صفحاتی از لیست قاب‌های آزاد که توسط هسته نگه‌داری می‌شوند تخصیص می‌یابند. این لیست با استفاده از یک الگوریتم جایگزینی صفحه، نظیر الگوریتم‌هایی که در بخش ۱۰-۴ مطرح شدند پر و خالی می‌شود و صفحات خالی آن احتمالاً همچنان که پیشتر تشریح شد در کل حافظه‌ی فیزیکی پراکنده شده‌اند. همچنین به یاد دارید که اگر پردازه‌ی کاربری درخواستی برای تخصیص یک

1. Univeral Windows Platform (UWP)

تک‌بایت هم داشته باشد، پارگی داخلی به وجود می‌آید، چرا که یک قاب‌صفحه یکجا به پردازه اعطا می‌شود.

با این وجود، حافظه‌ی هسته غالباً از یک انبار حافظه‌ی آزاد، جدا از لیستی که برای ارضای پردازه‌های عادی مد کاربر استفاده می‌شوند، تخصیص می‌یابد. دو دلیل اصلی برای انجام این کار وجود دارند که عبارتند از:

۱- هسته، حافظه را برای ساختمان‌داده‌هایی با اندازه‌های متغیر درخواست می‌کند که برخی از آنها کمتر از اندازه‌ی یک صفحه هستند. در نتیجه، هسته بایستی حافظه را محتاطانه استفاده کرده و سعی کند اتلاف حافظه را به خاطر پارگی کمینه سازد. دلیل اهمیت خاص این کار آن است که بسیاری از سیستم‌عامل‌ها، کد یا داده‌های هسته را به سیستم صفحه‌بندی عرضه نمی‌کنند.

۲- صفحاتی که به پردازه‌های مد کاربر تخصیص می‌یابند الزامی نیست که در حافظه‌ی فیزیکی همجوار باشند. با این وجود، دستگاه‌های سخت‌افزاری خاصی هستند که مستقیماً با حافظه‌ی فیزیکی تعامل دارند□بی آنکه نفعی از رابط حافظه‌ی مجازی ببرند□و در نتیجه ممکن است همجواری صفحات فیزیکی مقیم آنها ضروری باشد.

در بخش‌های بعدی، دو راهبرد مدیریت حافظه‌ی آزاد را برای تخصیص پردازه‌های هسته مرور می‌کنیم: «سیستم نیمه[1]» و تخصیص قالبی[2].

۱۰-۸-۱ سیستم نیمه

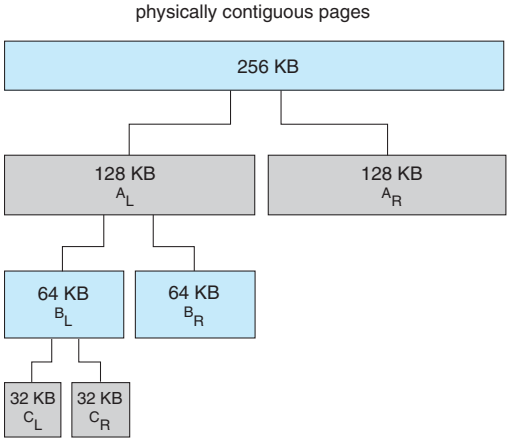
سیستم نیمه حافظه را از قطعه‌ای با اندازه‌ی ثابت شامل صفحات فیزیکی همجوار تخصیص می‌دهد. حافظه‌ی تخصیصی این قطعه از **تخصیص‌گر توان-۲** استفاده می‌کند که درخواست‌ها را به صورت واحدهایی از توان ۲ ارضا می‌نماید (۴ کیلوبایتی، ۸ کیلوبایتی، ۱۶ کیلوبایتی و الی آخر). درخواستی که اندازه‌ی واحد مناسبی نداشته باشد به بالاترین توان بعدی ۲ که بزرگ‌تر از درخواست خواهد بود، گرد می‌شود. برای مثال، یک درخواست برای ۱۱ کیلوبایت، با یک قطعه‌ی ۱۶ کیلوبایتی ارضا می‌شود.

حال مسأله را با یک مثال ساده مرور می‌کنیم. فرض کنید اندازه‌ی یک قطعه‌ی حافظه در ابتدا ۲۵۶ کیلوبایت است و هسته درخواستی برای ۲۱ کیلوبایت حافظه انجام می‌دهد. قطعه در ابتدا به دو **نیمه**—به آنها A_L و A_R خواهیم گفت—که اندازه‌های هر یک از آنها ۱۲۸ کیلوبایت می‌باشد، تقسیم می‌شود. یکی از این نیمه‌ها دوباره به دو نیمه—B_L و B_R—با اندازه‌های ۶۴ کیلوبایت تقسیم می‌شود. با این وجود، بالاترین توان بعدی ۲ بعد از ۲۱ کیلوبایت، ۳۲ کیلوبایت است، لذا B_L یا B_R، باز هم به دو نیمه‌ی ۳۲ کیلوبایتی C_L و C_R تقسیم می‌شوند. یکی از این نیمه‌ها برای ارضای درخواست ۲۱ کیلوبایتی استفاده می‌شود. این طرح در شکل ۱۰-۲۶ نشان داده می‌شود که در آن C_L قطعه‌ای است که به درخواست ۲۱ کیلوبایتی تخصیص می‌یابد.

1. Buddy system
2. Slab allocation

یکی از مزایای سیستم نیمه در سرعتی است که نیمه‌های مجاور می‌توانند با استفاده از تکنیکی موسوم به **ائتلاف**[1] با همدیگر ترکیب شده و قطعات بزرگتر را تشکیل بدهند. برای مثال، لحظه‌ای که هسته، در شکل ۱۰-۲۶، واحد C_L را که در اختیار دارد رها می‌کند، سیستم می‌تواند با ائتلاف C_L و C_R قطعه‌ی ۶۴ کیلوبایتی B_L را تشکیل بدهد. این قطعه نیز به نوبت خود می‌تواند با نیمه‌ی خودش یعنی B_R ائتلاف نماید تا قطعه‌ی ۱۲۸ کیلوبایتی را تشکیل بدهد. بالاخره، دوباره به قطعه‌ی ۲۵۶ کیلوبایتی اصلی خواهیم رسید.

عیب واضح سیستم نیمه این است که گردکردن درخواست‌ها به بالاترین توان بعدی ۲، احتمالاً در قطعات تخصیصی، پارگی به بار می‌آورد. برای مثال، یک درخواست ۳۳ کیلوبایتی را تنها می‌توان با یک قطعه‌ی ۶۴ کیلوبایتی ارضا کرد. به واقع نمی‌توانیم تضمین کنیم که کمتر از ۵۰ درصد واحد تخصیصی به خاطر پارگی داخلی به هدر نخواهد رفت. در بخش بعدی، یک طرح تخصیص حافظه را مطرح می‌کنیم که در آن هیچ‌گونه فضایی به خاطر پارگی از دست نمی‌رود.



شکل ۱۰-۲۶ تخصیص سیستم نیمه.

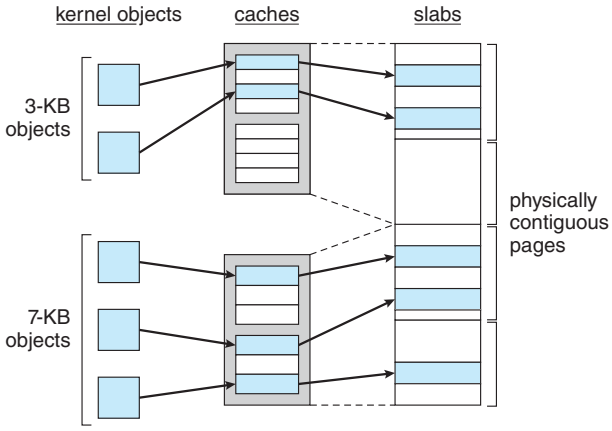
۱۰-۸-۲ تخصیص قالبی

راهبرد دومی برای تخصیص حافظه‌ی هسته وجود دارد که به **تخصیص قالبی** معروف است. هر **قالب** از یک یا چند صفحه‌ی فیزیکی همجوار ساخته می‌شود. **کَش** شامل یک یا چند قالب است. برای هر یک از ساختمان‌داده‌های هسته، کَش منفردی وجود دارد—برای مثال، یک کَش مجزا برای ساختمان‌داده‌های معرف توصیف‌گرهای پردازه، یک کَش مجزا برای اشیاء فایل، یک کَش مجزا برای سمافورها و الی آخر. هر کَش با **اشیائی** از نمونه‌های ساختمان‌داده‌های هسته که عرضه می‌دارد پر و خالی می‌شود. برای مثال، کَشی که برای سمافورها عرضه می‌شود نمونه‌هایی از اشیاء سمافور را ذخیره می‌نماید، کَشی که معرف توصیف‌گرهای پردازه است نمونه‌های اشیاء توصیف‌گرهای پردازه را ذخیره می‌سازد و الی آخر. رابطه‌ی میان قالب‌ها،

1. Coalescing

کَش‌ها و اشیاء در شکل ۱۰-۲۷ نشان داده می‌شود. شکل، دو شئ هسته با اندازه‌ی ۳ کیلوبایت و سه شئ با اندازه‌ی ۷ کیلوبایت را نشان می‌دهد که هر یک در یک کَش جداگانه ذخیره می‌شوند.

الگوریتم تخصیص-قالبی از کَش‌ها برای ذخیره‌ی اشیاء هسته استفاده می‌کند. موقعی که یک کَش ایجاد می‌شود، تعدادی شئ—که در ابتدا، علامت **آزاد** می‌خورند—به کَش تخصیص می‌یابند. تعداد اشیاء داخل کَش به اندازه‌ی قالب مرتبط بستگی دارد. برای مثال، یک قالب ۱۲ کیلوبایتی (مرکب از سه صفحه‌ی همجوار ۴ کیلوبایتی) می‌تواند شش شئ ۲ کیلوبایتی را ذخیره نماید. در ابتدا، تمامی اشیاء علامت آزاد می‌خورند. زمانی که شئ جدیدی برای یک ساختمان‌داده از هسته لازم می‌شود، تخصیص‌گر می‌تواند هر یک از اشیاء آزاد موجود کَش را برای ارضای درخواست بگمارد. شئ گماشته شده از کَش، علامت **استفاده** می‌خورد.



شکل ۱۰-۲۷ تخصیص قالبی.

بیایید شرایطی را در نظر بگیریم که در آن هسته از تخصیص‌گر قالب برای یک شئ معرف توصیف‌گر یک پردازه درخواست حافظه می‌نماید. توصیف‌گر پردازه در سیستم‌های لینوکس از نوع struct task_struct است که تقریباً به ۱/۷ کیلوبایت حافظه نیاز دارد. لحظه‌ای که هسته‌ی لینوکس وظیفه‌ی جدیدی ایجاد می‌کند، حافظه‌ی مورد نیاز شئ struct task_struct را از کَش آن درخواست می‌نماید. کَش، این درخواست را با استفاده از شئ struct task_struct که پیشاپیش از یک قالب تخصیص پیدا کرده و علامت آزاد خورده است، به انجام می‌رساند.

در لینوکس، هر قالب می‌تواند در یکی از سه حالت ممکن زیر باشد:

۱- **پر**: تمام اشیاء موجود قالب علامت استفاده خورده‌اند.
۲- **خالی**: تمامی اشیاء موجود قالب علامت آزاد خورده‌اند.
۳- **نیمه‌پر**: قالب شامل هر دوی اشیاء استفاده شده و اشیاء آزاد است.

تخصیص‌گر قالبی نخست سعی می‌کند که درخواست را با یک شئ آزاد از یک قالب نیمه‌پر ارضا نماید. اگر چنین قالبی وجود نداشت، شئ آزادی را از یک قالب خالی برای ارضای

درخواست می‌گمارد. اگر قالب خالی هم نباشد، قالب جدیدی از صفحات فیزیکی همجوار تخصیص یافته و به یک کَش گمارده می‌شود؛ حافظه‌ی مورد نیاز شئ از همین قالب تخصیص می‌یابد.

تخصیص‌گر قالبی دو فایده‌ی مهم دارد که عبارتند از:

۱- هیچ‌گونه حافظه به خاطر پارگی هدر نمی‌رود. پارگی نمی‌تواند مسأله باشد چرا که به هر یک از ساختمان‌داده‌های هسته کَش یکتایی منتسب می‌شود و هر کَش مرکب از یک یا چند قالب است که به واحدهایی از اندازه‌های اشیاء موجود هسته تقسیم می‌شوند. بنابراین، لحظه‌ای که هسته برای یک شئ درخواست حافظه می‌کند، تخصیص‌گر قالبی دقیقاً همان میزان حافظه را تخصیص می‌دهد که شئ مورد نظر لازم دارد.

۲- درخواست‌های حافظه می‌توانند به سرعت ارضا شوند. طرح تخصیص قالبی مخصوصاً زمانی برای مدیریت حافظه سودمند خواهد بود که اشیائی به کرات تخصیص و رها شوند که این امر غالباً با درخواست‌های هسته اتفاق می‌فتد. عمل تخصیص و رهایی حافظه می‌تواند فرایند وقت‌گیری باشد. با این وجود، در اینجا چون اشیاء از قبل ایجاد می‌شوند تخصیص آنها سریعاً از کَش انجام می‌گیرد. به علاوه، زمانی که کار هسته با یک شئ تمام می‌شود و آن را رها می‌سازد، علامت آزاد می‌خورد و به کَش خود برگردانده می‌شود که با این کار، بلافاصله می‌تواند برای ارضای درخواست‌های بعدی هسته به کار گرفته شود.

تخصیص‌گر قالبی برای نخستین بار خود را در هسته‌ی سولاریس 2.4 نشان داد. به خاطر طبیعت همه‌منظوره‌ای که این تخصیص‌گر دارد، هنوز هم برای برخی درخواست‌های خاص حافظه در مد کاربر سولاریس استفاده می‌شود. لینوکس در ابتدا از سیستم نیمه استفاده می‌کرد؛ با این وجود، با شروع نسخه‌ی 2.2 هسته‌ی خود را با تخصیص‌گر قالبی سازگار نمود. لینوکس به پیاده‌سازی قالبی خود، SLAB اطلاق می‌کند. توزیع‌های اخیر لینوکس، دو تخصیص‌گر دیگر– تخصیص‌گرهای SLOB و SLAB–را برای حافظه‌ی هسته کار گذاشته‌اند.

تخصیص‌گر SLOB برای سیستم‌هایی با یک میزان حافظه‌ی محدود، همچون سیستم‌های تعبیه‌شده طراحی می‌شود. SLOB (که اختصار «لیست ساده‌ی بلاک‌ها[1]» می‌باشد) سه لیست شئ نگه می‌دارد: **کوچک** (برای اشیاء کوچک‌تر از ۲۵۶ بایت)، **متوسط** (برای اشیاء کوچک‌تر از ۱۰۲۴ بایت) و **بزرگ** (برای تمامی اشیاء دیگر کوچک‌تر از اندازه‌ی یک صفحه). درخواست‌های حافظه‌ی یک شئ با یک اندازه‌ی لیست مناسب با استفاده از یک سیاست اولین-گنجایش تخصیص داده می‌شوند.

با شروع نسخه‌ی 2.6.24، تخصیص‌گر SLUB به عنوان تخصیص‌گر پیش‌فرض هسته‌ی لینوکس، جایگزین SLAB شد. SLUB، بیشتر سربار لازم تخصیص‌گر SLAB را کاهش داد. برای نمونه، در حالی که SLAB ابرداده‌های خاصی را با هر قالب ذخیره می‌کند، SLUB این داده‌ها را در ساختار page که هسته‌ی لینوکس برای هر صفحه استفاده می‌کند ذخیره می‌سازد. به علاوه،

1. Simple list of blocks (SLOB)